

Parcial 2 – Prueba 2

Ejercicio 1

Deberás completar el método auxiliar recursivo **validarABBRec()** y el método público **validarABB()** para garantizar que se cumple las condiciones de un Arbol binario de búsqueda.

La definición de un ABB dice que para cualquier nodo p:

- Todos los valores en el subárbol izquierdo de p deben ser menores que p->dato.
- Todos los valores en el subárbol derecho de p deben ser mayores que p->dato.

TIP: Ambas condiciones establecen que TODOS los nodos del sub arbol tienen que ser menores o mayores respectivamente.

Código base:

```
class Nodo {
public:
    int dato;
    Nodo* izq;
    Nodo* der;
    explicit Nodo(const int& x) : dato(x), izq(nullptr), der(nullptr) {}
};

class ABB {
private:
    Nodo* insertarRec(Nodo* p, const int& x) {
        if (!p) {
            return new Nodo(x);
        }
        if (x < p->dato) {
            p->izq = insertarRec(p->izq, x);
        } else if (x > p->dato) {
            p->der = insertarRec(p->der, x);
        }
        return p;
    }

    bool validarABBRec(const Nodo* p, const int* minVal, const int* maxVal)
const ;

public:
    Nodo* raiz = nullptr; // Para poder generar escenario con ABB invalidos

    ABB() = default;

    void insert(const int& x) {
        raiz = insertarRec(raiz, x);
    }

    bool validarABB() const ;
};
```

Ejercicio 2 -

Estás desarrollando el software para un sistema embebido distribuido (por ejemplo, una red IoT de sensores industriales).

Cada nodo del sistema es un módulo (sensor, actuador, controlador) y está conectado a otros mediante enlaces directos (por ejemplo, bus CAN o I2C).

Debes representar esta red como un **grafo no dirigido**, donde:

- Cada módulo es un nodo.
- Las conexiones físicas entre ellos son aristas.

Tu tarea:

Implementa una clase `RedDeSensores` que permita:

1. **Agregar conexiones** entre módulos.
2. **Verificar si dos módulos están directamente conectados** (de forma iterativa).
3. **Calcular cuántos módulos son alcanzables** desde un nodo dado (usando DFS recursivo).

```
#include <iostream>
```

```
#include <vector>
```

```
class RedDeSensores {
```

```
public:
```

```
    RedDeSensores(int cantidadModulos) {
```

```
        // Implementar
```

```
    }
```

```
    void agregarConexion(int a, int b) {
```

```
        // Implementar
```

```
    }
```

```
    void imprimirConexion(int a, int b) {
```

```
        // Implementar
```

```
        // Salida esperada Conexion directa (<a> - <b>): <cantidad>
```

```
    }
```

```
    void imprimirAlcanzables(int desde) {
```

```
        // Implementar
```

```
        // Salida esperada Modulos alcanzables desde <desde>: <cantidad>
```

```
    }
```

```
private:
```

```
    int cantidad;
```

```
    std::vector<std::vector<int>>> adyacencias;
```

```
    // Agregar estructuras internas necesarias
```

```
};
```

Ejercicio 3 – Teorico

¿Cuál de las siguientes afirmaciones describe mejor el concepto de "función hash" en el contexto de una tabla hash?

Pregunta 3 Respuesta

a.

Una función que convierte una clave de entrada en un índice (posición) en el arreglo de la tabla hash.

b.

Un algoritmo para ordenar los elementos dentro de la tabla hash.

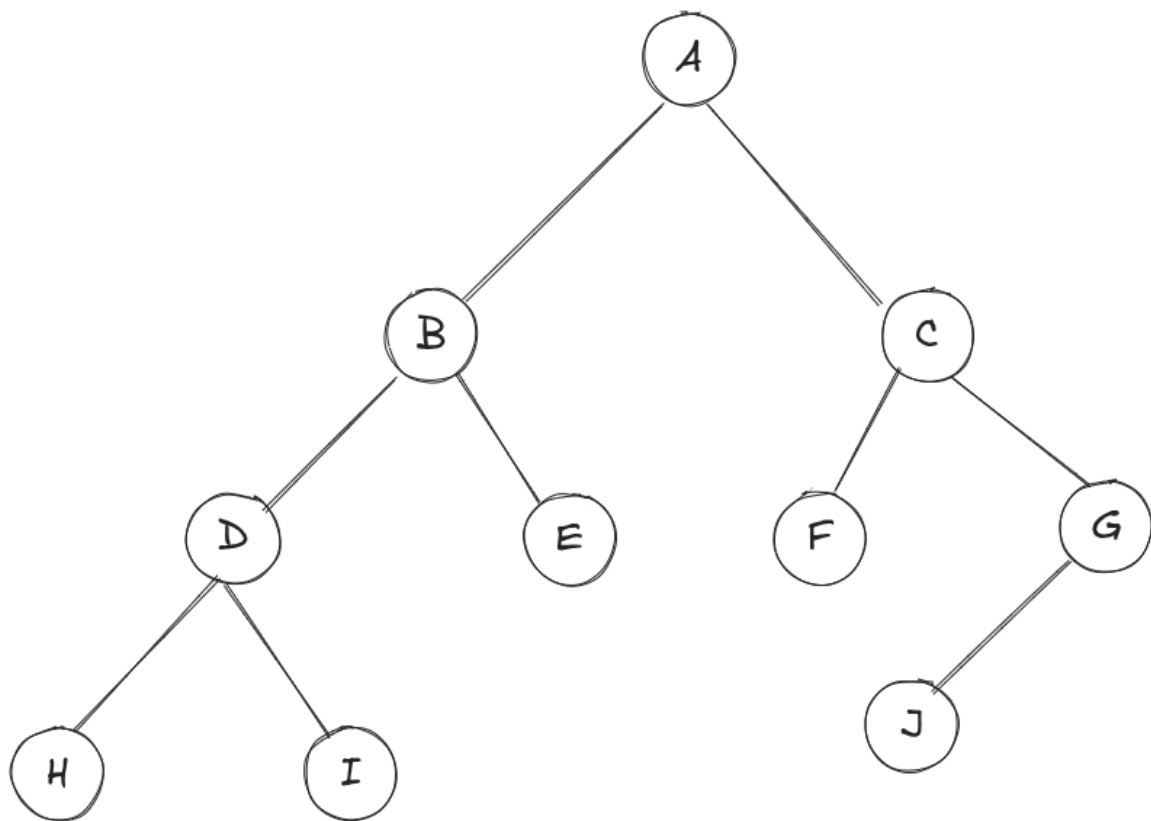
c.

Un método para comprimir los datos antes de almacenarlos.

d.

Una forma de garantizar que no ocurran colisiones en la tabla.

Ejercicio 4 – Teorico



El recorrido **inorden**, los nodos recorridos son...

Pregunta 4 Respuesta

a.

[H, D, I, B, E, A, F, C, J, G]

b.

[A, B, D, H, I, E, C, G, F, J]

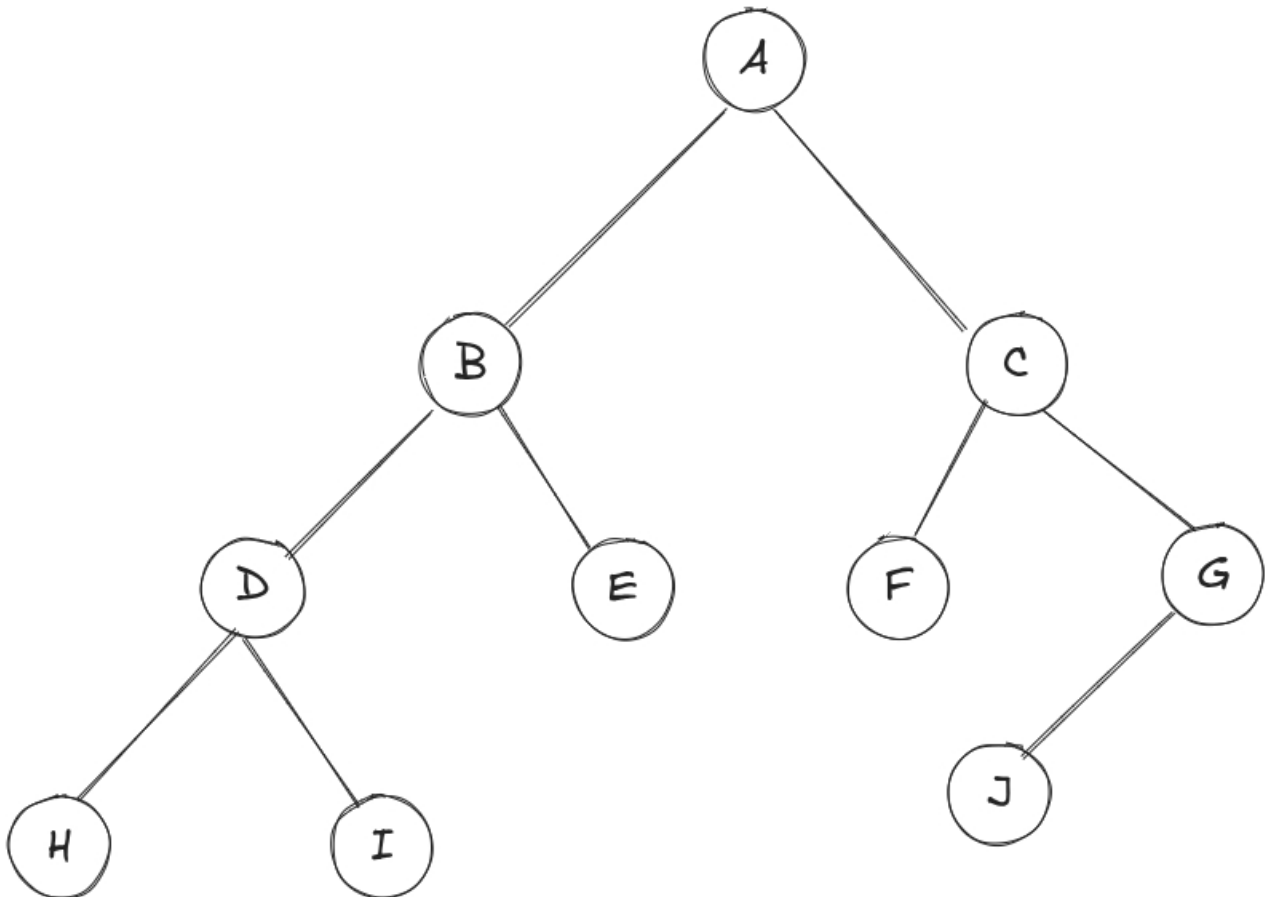
c.

[A, B, D, H, I, E, C, F, G, J]

d.

[D, H, I, B, E, A, F, C, J, G]

Ejercicio 5 – Practico



El recorrido **postorden**, los nodos recorridos son...

Pregunta 5 Respuesta

a.

[A, B, D, H, I, E, C, F, G, J]

b.

[H, I, D, E, B, F, J, G, C, A]

c.

[H, D, I, B, E, A, F, C, J, G]

d.

[D, H, I, B, E, A, F, C, J, G]